Nome e Cognome: _____ **Matricola:** _____

Istruzioni: Il tempo a disposizione per completare la prova è di **45 minuti**. Durante la prova, non è consentito consultare materiale didattico, né interagire con dispositivi elettronici o colleghi. Nel caso di domande a risposta multipla con più risposte corrette, ciascuna risposta corretta indicata vale una corrispondente frazione del punteggio complessivo, mentre le risposte errate comportano una pari penalizzazione. In nessun caso, il punteggio complessivo relativo a una domanda sarà negativo.

Es. **1.** **(2 punti)** Che cosa indica il codice di stato `404` in una risposta HTTP?

- [ ] A La copia in cache del client è ancora valida (`Not Modified`).
- [ ] B Il metodo non è ammesso per quella risorsa (`Method Not Allowed`).
- [x] **C** La risorsa richiesta non esiste (`Not Found`).
- [ ] D La risorsa si è spostata in modo permanente a un altro indirizzo.

Risposta: C. `404` appartiene alla classe 4xx. Promemoria: 200 OK, 201 Created, 204 No Content; 301 spostamento permanente, 302 temporaneo, 304 «la tua copia in cache va ancora bene»; 400 richiesta malformata, 401 «non so chi sei», 403 «so chi sei, ma non puoi», 404 risorsa inesistente, 405 metodo non ammesso; 500 errore interno, 503 server non disponibile. *(Dispensa: cap. 1, «Codici di stato della risposta»; cap. 9, «La risposta»)*

Es. **2.** **(2 punti)** Quali dei seguenti elementi HTML sono **elementi vuoti** (*void*), cioè non hanno contenuto né tag di chiusura?

- [ ] A `<a>`
- [ ] B `<script>`
- [x] **C** `<meta>`
- [ ] D `<li>`
- [x] **E** `<br>`
- [x] **F** `<link>`

Risposte: C, E, F. Gli elementi vuoti hanno solo il tag di apertura (con eventuali attributi): `br`, `img`, `input`, `meta`, `link`, `hr`, … `<script>` invece va sempre chiuso con `</script>`, anche quando usa l'attributo `src`. *(Dispensa: cap. 2, «Tag, elementi e attributi»)*

Es. **3.** **(2 punti)** Quali delle seguenti proprietà vengono **ereditate** per impostazione predefinita dagli elementi figli?

- [x] **A** `color`
- [ ] B `width`
- [x] **C** `font-size`
- [ ] D `border`
- [ ] E `background-color`
- [x] **F** `text-align`

Risposte: A, C, F. Si ereditano soprattutto le proprietà legate al testo (colore, font, allineamento); quelle del riquadro (bordi, margini, padding, sfondo, dimensioni) no. Con il valore `inherit` si può forzare l'ereditarietà di qualunque proprietà. *(Dispensa: cap. 3, «Ereditarietà»)*

Es. **4.** **(2 punti)** Che cosa distingue un layout **responsive** da uno **adattivo**?

- [x] **A** Il responsive si adatta con continuità a ogni larghezza; l'adattivo usa alcune larghezze fisse.
- [ ] B L'adattivo funziona solo con JavaScript.
- [ ] C Il responsive usa solo unità assolute come `px`.
- [ ] D L'adattivo si adatta con continuità; il responsive usa alcune larghezze fisse.
- [ ] E Il responsive richiede un sito separato per i dispositivi mobili.
- [ ] F Nessuna: sono due nomi della stessa tecnica.

Risposta: A. Un layout adattivo prevede alcuni layout a larghezza fissa e sceglie il più adatto; un layout responsive combina dimensioni fluide (percentuali, `fr`, Flexbox) e media query per adattarsi a qualunque larghezza. Esistono poi i layout fissi, quelli liquidi e i siti mobili separati. *(Dispensa: cap. 4, «Responsive design»)*

SOLUZIONI

Es. **5.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
1  const pet = {
2    nome: "Fido",
3    saluta() { console.log("Ciao, sono " + this.nome); }
4  };
5  pet.saluta();
6  const f = pet.saluta;
7  f();
```

- [ ] A — Viene sollevato un `TypeError`.
- [x] B — `Ciao, sono Fido`, poi viene sollevato un `TypeError`.
- [ ] C — `Ciao, sono Fido, Ciao, sono Fido.`
- [ ] D — `Ciao, sono Fido, Ciao, sono .`

Risposta: B. `this` dipende da **come** si chiama la funzione: in `pet.saluta()` vale `pet`. `f` è la stessa funzione chiamata da sola: in modalità strict `this` vale `undefined`, e leggere `this.nome` solleva un `TypeError`. *(Dispensa: cap. 5, «La parola chiave this»)*

Es. **6.** **(2 punti)** Qual è il risultato dell'esecuzione del seguente frammento di codice JavaScript? Si assuma la presenza di `"use strict";`.

```
1  function conta(primo, ...altri) {
2    return altri.length;
3  }
4  const v = [1, 2];
5  console.log(conta(...v, 3, ...v));
6  const o = { a: 1, b: 2 };
7  const p = { ...o, b: 3, c: 4 };
8  console.log(Object.values(p).join(","));
```

- [ ] A — 5, `1,3,4`.
- [x] B — 4, `1,3,4`.
- [ ] C — 2, `1,3,4`.
- [ ] D — 4, `1,2,3,4`.
- [ ] E — 3, `1,3,4`.
- [ ] F — 4, `1,2,4`.

Risposta: B. Nella chiamata lo spread espande gli array: gli argomenti sono 1, 2, 3, 1, 2; il parametro *rest* `...altri` raccoglie tutti quelli dopo il primo (4). Negli oggetti lo spread copia le proprietà, e quelle scritte dopo sovrascrivono quelle con lo stesso nome. *(Dispensa: cap. 6, «Assegnamenti destrutturanti»)*

Es. **7.** **(2 punti)** Quali delle seguenti affermazioni su npm e sul file `package.json` sono **vere**?

- [ ] A — `"main"` indica il comando da eseguire con `npm start`.
- [ ] B — `package.json` contiene il codice dell'applicazione.
- [ ] C — Le dipendenze vengono installate nella cartella del sistema, comune a tutti i progetti.
- [ ] D — `npm install` scarica solo il pacchetto richiesto, non le sue dipendenze.
- [ ] E — `node_modules` va versionata, perché senza non si può ricostruire il progetto.
- [x] F — `"type": "module"` abilita la sintassi `import`/`export` dei moduli ES.

Risposta: F. `package.json` descrive il pacchetto (come il `pom.xml` di Maven): nome, versione, file principale (`main`), tipo di moduli, attività (`scripts`) e dipendenze. npm scarica le dipendenze, ricorsivamente, in `node_modules`, che si ricostruisce da `package.json` e per questo resta fuori dal controllo di versione. *(Dispensa: cap. 11, «npm: il gestore di pacchetti»)*

SOLUZIONI

Es. **8.** **(2 punti)** Si considerino le regole seguenti, applicate a un `div` di classe `elemento` contenuto in un `div` di classe `contenitore`. Quanto è largo l'elemento (con `1rem` pari a 16px)?

```
.contenitore {
  width: 300px;
}
.elemento {
  width: calc(100% - 20px);
}
```

- A 300px
- B 320px
- C 150px
- D 260px
- E 20px
- **F** 280px

Risposta: F. Nella proprietà `width` le percentuali si riferiscono alla larghezza del contenitore (300px). `min()` e `max()` scelgono il più piccolo e il più grande degli argomenti, `clamp(minimo, preferito, massimo)` vincola il valore preferito fra i due estremi. Dentro `calc()` gli operatori `+` e `-` vanno circondati da spazi: senza, la dichiarazione non è valida e viene scartata, e il `div` resta largo quanto il contenitore. *(Dispensa: cap. 14, «Valori calcolati»)*

Es. **9.** **(5 punti)** Che cosa sono il DOM e il BOM? Si descriva il ruolo dell'oggetto `window` e si facciano esempi di oggetti e operazioni di ciascuno.

Traccia della risposta.

- `window` è l'oggetto globale dell'ambiente del browser (funzioni e variabili `var` globali ne diventano proprietà) e rappresenta la finestra del browser.
- Il DOM rappresenta il contenuto del documento come un albero di nodi (elementi, testo, commenti) con radice `document`: serve a leggere e modificare la pagina (`querySelector`, `textContent`, `append`...).
- Il BOM comprende gli oggetti per lavorare con il browser stesso: `location` (URL), `history` (cronologia), `navigator` (browser e piattaforma), `screen` (schermo).
- Entrambi sono forniti dall'ambiente ospite, non dal linguaggio: in Node.js non esistono.

(Dispensa: cap. 7, «L'ambiente del browser»)

Es. **10.** **(5 punti)** Si descriva il modello di esecuzione di JavaScript nel browser (thread, Web API, coda dei callback, event loop), spiegando con un esempio perché `setTimeout(f, 0)` non esegue `f` immediatamente.

Traccia della risposta.

- Il codice della pagina gira su un solo thread, che esegue script, gestori di eventi e aggiornamento della pagina: un'attesa sincrona blocca tutto.
- Le operazioni lente (timer, richieste di rete, eventi) sono gestite dal browser tramite le Web API: la chiamata ritorna subito e l'attesa procede fuori dal thread.
- Al termine il browser mette la callback nella coda dei callback; l'event loop la esegue quando lo stack delle chiamate è vuoto.
- Es.: `console.log("A"); setTimeout(() => console.log("B"), 0); console.log("C");` stampa A, C, B. Asincrono non vuol dire parallelo.

(Dispensa: cap. 8, «Sincronismo e asincronismo»)

Es. **11.** **(5 punti)** Che differenza c'è fra una libreria e un framework? Si spieghi l'inversione del controllo e si descriva che cosa offre tipicamente un web framework, con riferimento a Express.

Traccia della risposta.

- Libreria: funzioni che il nostro codice chiama, e che restituiscono il controllo; il flusso lo decide il nostro codice.
- Framework: progetto astratto in cui il nostro codice viene innestato; è il framework a controllare il flusso e a chiamare il nostro codice (IoC, «non chiamarci, ti chiamiamo noi»).
- Un web framework offre routing, analisi delle richieste, cookie e sessioni, un motore di template, spesso un ORM; può essere più o meno *opinionated*.
- Express: minimalista e unopinionated; rotte (`app.get(percorso, gestore)`, parametri, router), middleware, `res.render` con un motore di template.

(Dispensa: cap. 15, «Che cos'è un framework»)

Griglia delle risposte

Es.	Risposta	Capitolo	Argomento
1	C	1	codici di stato: i singoli codici
2	C, E, F	2	tag, elementi e attributi
3	A, C, F	3	ereditarietà
4	A	4	responsive design
5	B	5	this e costruttori
6	B	6	destrutturazione e spread
7	F	11	npm e package.json
8	F	14	valori calcolati
9	aperta	7	DOM e BOM
10	aperta	8	event loop e callback
11	aperta	15	framework

Punteggio massimo: 16 (chiuse) + 15 (aperte) = **31 punti**. Nelle domande con più risposte corrette ogni risposta giusta indicata vale una frazione dei punti e ogni risposta sbagliata indicata ne toglie altrettanti; il punteggio di una domanda non scende mai sotto zero.

SOLUZIONI